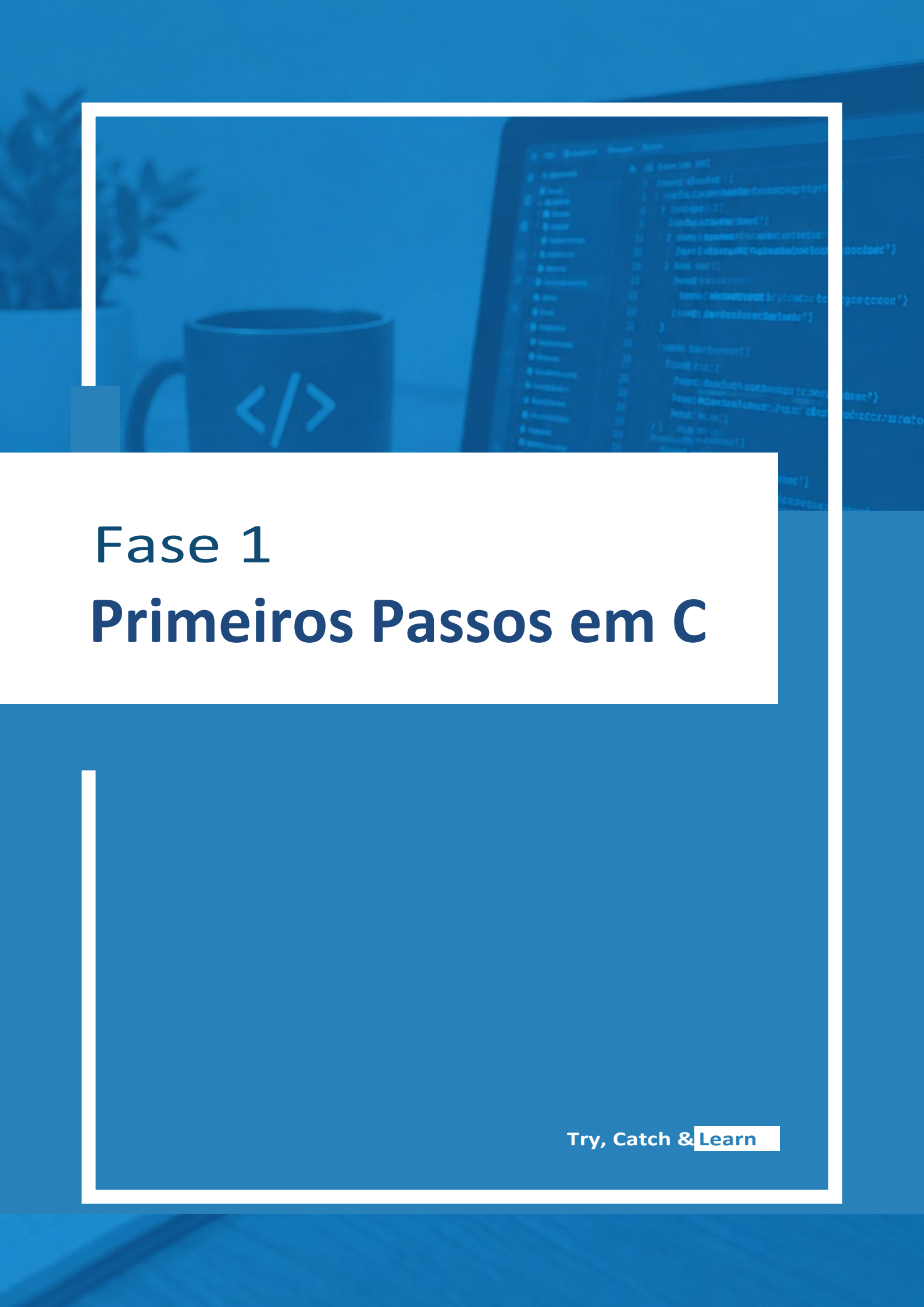




Fase 1

Primeiros Passos em C

Try, Catch & Learn

Fase 1

Primeiros Passos em C

Antes de ensinar o programa a decidir, vamos ensiná-lo a existir: ler, guardar, calcular e mostrar informação

1. O que é programação?

Pense numa receita:

1. Pegar os ingredientes;
2. Misturar;
3. Colocar no forno;
4. Esperar;
5. Servir.

- Um programa é isso: uma sequência de instruções, executadas uma depois da outra, para chegar a um resultado. Programar é escrever essa receita numa linguagem que o computador entende.
- Programar é dar instruções para o computador executar, passo a passo, até resolver uma tarefa. O computador não tem iniciativa própria: ele faz exatamente o que escrevemos, na ordem em que escrevemos.
- A ordem das instruções importa tanto quanto as próprias instruções. Guarde essa ideia: programa = sequência de instruções.

2. O que é a linguagem C?

C é uma linguagem de programação criada nos anos 1970, ainda hoje muito usada para sistemas operacionais, softwares embarcados e programas que precisam de desempenho e maior proximidade com o funcionamento do computador. Para quem está aprendendo lógica de programação, C é uma boa escolha porque é uma linguagem direta e ajuda a perceber os passos que o programa está executando.

Não se preocupe agora com compiladores, memória ou arquitetura do computador. Por enquanto, C é apenas a linguagem que vamos usar para escrever nossas instruções.

3. O primeiro programa em C

Todo programa em C começa com uma estrutura parecida com esta:

```
#include <stdio.h>  
int main() {  
    printf("Ola, mundo!\n");  
    return 0;  
}
```

Vamos entender cada parte:

#include <stdio.h> — informa ao programa que queremos incluir a biblioteca e

utilizar as funções que ela disponibiliza.

int main() — é a função principal do programa. A execução começa por ela.

{ } — delimitam o bloco de instruções da função ***main***.

printf() — mostra informações na tela.

\n — indica uma quebra de linha.

return 0; — encerra a função ***main*** indicando que o programa terminou normalmente.

Não é preciso decorar essa estrutura agora. Ela vai aparecer em praticamente todos os programas que você escrever e ficará natural com a prática.

4. Instruções e sequência de execução

O computador executa as instruções na ordem em que elas aparecem no código, uma por vez, de cima para baixo.

```
printf("Primeiro\n");  
printf("Segundo\n");  
printf("Terceiro\n");
```

Saída:

```
Primeiro  
Segundo  
Terceiro
```

O programa ainda não está tomando decisões. Ele apenas segue, em ordem, cada instrução que escrevemos. Essa previsibilidade será a base para os próximos assuntos.

5. Variáveis: guardando informações

Uma variável pode ser imaginada como uma caixa com um nome. Ela guarda um valor que o programa pode usar, mostrar ou alterar depois.

```
int idade = 20;  
float altura = 1.65;  
char inicial = 'L';
```

Em cada declaração temos um tipo, um nome e um valor.

Declaração é avisar que a variável existe. Inicialização é colocar um valor nela logo de início.

Por enquanto, não precisamos entender onde fisicamente essa informação fica guardada, a ideia de "caixa com nome e valor" é suficiente.

6. Tipos básicos de dados

O tipo de uma variável indica que tipo de valor ela pode guardar. Neste momento, quatro tipos são suficientes:

Tipo	Representa	Exemplo
int	números inteiros	20
float	números decimais	1.65
double	números decimais com maior precisão	10.50
char	um único caractere	'L'

A linguagem C não possui um tipo nativo chamado string para textos inteiros, os textos são tratados de outra forma, que será estudada mais adiante.

7. Mostrando informações com printf

Além de mostrar textos fixos, printf pode mostrar o valor de uma variável usando um marcador de posição.

```
int idade = 20;  
printf("Idade: %d\n", idade);
```

%d - indica que naquele ponto será colocado um número inteiro. Depois da vírgula, informamos a variável que preencherá esse espaço.

Marcador	Usado para
%d	números inteiros (int)
%f	números decimais (float / double)
%c	um caractere (char)

```
float altura = 1.65;  
char inicial = 'L';  
printf("Altura: %f\n", altura);  
printf("Inicial: %c\n", inicial);
```

8. Recebendo informações com scanf

Até agora, os valores das variáveis vieram prontos no código. Um programa fica mais útil quando pode receber informações do usuário.

```
int idade;
```

```
printf("Digite sua idade: ");
scanf("%d", &idade);
printf("Sua idade é: %d\n", idade);
```

scanf pausa o programa, espera o usuário digitar algo e guarda esse valor em uma variável. O **%d** indica que estamos esperando um número inteiro.

O **&** antes de **idade** significa **"endereço de memória de"**. Ele informa ao **scanf** onde a variável está armazenada na memória, para que o valor digitado possa ser colocado naquele local.

Regra prática: ao usar **scanf** para ler uma variável, coloque **&** antes do nome da variável.

9. Operações matemáticas

Antes de ensinar o programa a decidir, vamos ensiná-lo a calcular. Os operadores básicos são: + soma, - subtração, * multiplicação, / divisão, % resto da divisão

```
int a = 10;
int b = 3;
printf("%d\n", a + b); // 13
printf("%d\n", a - b); // 7
printf("%d\n", a * b); // 30
printf("%d\n", a / b); // 3
printf("%d\n", a % b); // 1
```

Repare na divisão: 10 / 3 resulta em 3, e não em 3,33. Quando os dois valores são int, C faz divisão inteira e descarta a parte decimal.

O operador % devolve o resto da divisão. Em 10 dividido por 3, o resto é 1; por isso, 10 % 3 resulta em 1.

$$\begin{array}{r|l} 10 & 3 \\ \hline 9 & 3 \\ \hline 1 & \leftarrow \text{resto da divisão} \end{array}$$

10. Pensando como um programador

Antes de escrever qualquer linha de código, um programador pensa no problema em etapas. Essa é uma das partes mais importantes da lógica de programação.

Problema: calcular o preço total de dois produtos.

Raciocínio:

1. receber o preço do produto 1
2. receber o preço do produto 2
3. somar os dois valores
4. mostrar o resultado

Agora, o mesmo raciocínio em C:

```
#include <stdio.h>
float produto1, produto2, total;
printf("Preço do produto 1: ");
scanf("%f", &produto1);
printf("Preço do produto 2: ");
scanf("%f", &produto2);
```

```
total = produto1 + produto2;
printf("Total: %f\n", total);
```

Outro exemplo: calcular a média de três notas.

Raciocínio:

1. receber nota 1
2. receber nota 2
3. receber nota 3
4. somar as notas
5. dividir por 3
6. mostrar a média

```
#include <stdio.h>
float n1, n2, n3, media;
    printf("Nota1: ");
    scanf("%f", &n1);

printf("Nota 2: ");
scanf("%f", &n2);

printf("Nota 3: ");
scanf("%f", &n3);
media = (n1 + n2 + n3) / 3;

printf("Media: %f\n", media);
```

O caminho é sempre o mesmo: problema → passos → código. Escrever os passos em português antes de programar ajuda a organizar o raciocínio e evitar erros.

11. Primeiro exercício de lógica

Use apenas o que já foi visto: variáveis, printf, scanf e operações matemáticas. Para cada exercício, primeiro escreva o raciocínio em passos. Só depois tente transformar esses passos em código.

Exercício 1 — Soma

Receba dois números e mostre a soma.

Exercício 2 — Idade

Receba o ano de nascimento e o ano atual e calcule a idade aproximada.

Exercício 3 — Média

Receba três notas e calcule a média.

Exercício 4 — Conversão

Receba uma temperatura em Celsius e converta para Fahrenheit.

Dica: $F = C \times 1.8 + 32$

12. O que você já consegue fazer

Ao terminar esta aula, você já consegue:

- escrever e ler um programa C básico, entendendo o papel de cada parte da estrutura;
- criar variáveis para guardar números inteiros, decimais e caracteres;

- mostrar valores na tela com `printf` e ler valores digitados com `scanf`;
- realizar cálculos com `+`, `-`, `*`, `/` e `%`;
- pensar em um problema em etapas antes de escrever o código.

Próxima aula

Agora que já sabemos guardar informações, receber dados e realizar operações, podemos ensinar o programa a tomar decisões.